

Estruturas de Dados: Introdução



Estruturas de Dados

Edson da Silva Castro

IFMS

Instituto Federal de Educação Ciência e Tecnologia de Mato Grosso do Sul

Três Lagoas, 2024

Algoritmos

Um Algoritmo pode ser definido como uma sequência ordenada de passos executáveis para resolver um problema.



Algoritmos

Segundo Castro(2020)

" Os algoritmos apresentam-se como ferramentas importantes para solução de problemas computacionais, permitindo tanto a especificação de soluções para estes problemas quanto a criação de novos conceitos."

" O aprendizado de algoritmos é essencial para que o estudante ... desenvolva as habilidades necessárias para pensar e se expressar tecnicamente de forma clara e precisa."

Algoritmos

Segundo Castro(2020)

"... (algoritmos permitem) a programação de computadores e de outros dispositivos eletrônicos. Neste sentido, a correta especificação de algoritmos mostra-se como premissa para o desenvolvimento de programas, pois preestabelecem e organizam a solução de problemas computacionais."

Falta Algo :(



:(

Até o momento provavelmente você desenvolveu algoritmos e programas sem se preocupar com estruturas de dados.

Estruturas de Dados

Importância das Estruturas de Dados

Segundo Backes(2016) diversas áreas do conhecimento se valem das estruturas de dados para diferentes propósitos. Ele afirma que a aplicação conjunta de **algoritmos e estruturas de dados** são de **"extrema importância na Ciência da Computação"** (grifo nosso).

Estruturas de Dados

Definição

"Em computação, uma estrutura de dados é uma forma de armazenar e organizar os dados de modo que possam ser usados de forma eficiente. Uma estrutura de dados é um relacionamento lógico entre diferentes tipos de dados visando à resolução de determinado problema de forma eficiente" (BACKES, 2016).

"Estrutura de dados é o ramo da computação que estuda os diversos mecanismos de organização de dados para atender aos diferentes requisitos de processamento (RICARTE, 2008)."

Estruturas de Dados

Tanto Backes(2016) quanto Ricarte(2008) concordam que a utilização de estruturas de dados adequadas para cada situação permitem a construção de algoritmos mais performáticos bem como a construção de aplicações altamente especializadas.

Constatação

Aplicações diferentes requerem diferentes estruturas de dados e diferentes algoritmos!

Organização da Memória

Durante a execução de um programa a memória do computador é organizada nos seguintes:

- **Código executável:** código binário do programa;
- **Dados estáticos:** variáveis globais e estáticas, que permanecem durante toda a execução do programa;
- **Pilha:** variáveis locais, que são criadas (empilhadas) na execução de uma função e liberadas (desempilhadas) ao término da função;
- **Heap:** variáveis alocadas dinamicamente.

Pilha versus Heap

Quantidade de memória

- **Pilha:** O sistema operacional limita a quantidade de memória passível de alocação na pilha de execução das funções;
- **Heap:** Pode-se alocar toda a memória ainda disponível no computador.

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha
- Armazena dados locais e endereços de retorno para passagem de parâmetros

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha
- Armazena dados locais e endereços de retorno para passagem de parâmetros
- Os dados podem ser usados sem ponteiros

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha
- Armazena dados locais e endereços de retorno para passagem de parâmetros
- Os dados podem ser usados sem ponteiros
- Pode ocorrer estouro de pilha

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha
- Armazena dados locais e endereços de retorno para passagem de parâmetros
- Os dados podem ser usados sem ponteiros
- Pode ocorrer estouro de pilha
- Adequada para se utilizar quando se sabe exatamente o quanto de espaço será alocado antes de compilar, e esse espaço não é muito grande

Alocação estática na pilha (stack) - BACKES(2016)

- Armazenado na memória RAM
- Variáveis são liberadas automaticamente no final do escopo
- Alocação mais rápida que na heap
- Utiliza uma estrutura de dados do tipo pilha
- Armazena dados locais e endereços de retorno para passagem de parâmetros
- Os dados podem ser usados sem ponteiros
- Pode ocorrer estouro de pilha
- Adequada para se utilizar quando se sabe exatamente o quanto de espaço será alocado antes de compilar, e esse espaço não é muito grande
- Geralmente possui um tamanho máximo predeterminado quando o programa inicia

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**
- Pode sofrer fragmentação após sucessivas alocações e liberações de memória

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**
- Pode sofrer fragmentação após sucessivas alocações e liberações de memória
- **Os dados são acessados por ponteiros**

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**
- Pode sofrer fragmentação após sucessivas alocações e liberações de memória
- **Os dados são acessados por ponteiros**
- A alocação pode falhar se muita memória for solicitada

Alocação dinâmica na heap - BACKES(2016)

- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**
- Pode sofrer fragmentação após sucessivas alocações e liberações de memória
- **Os dados são acessados por ponteiros**
- A alocação pode falhar se muita memória for solicitada
- Adequada para se utilizar quando **não se sabe exatamente o quanto de espaço será alocado** antes de compilar, **ou esse espaço a alocar for muito grande**

Alocação dinâmica na heap - BACKES(2016)

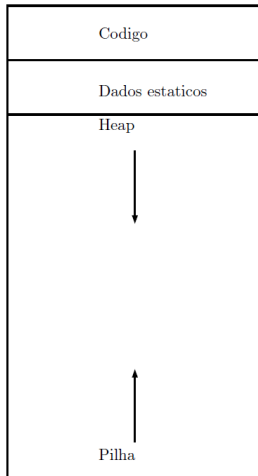
- Armazenado na memória **RAM**
- **Variáveis não dependem do escopo** (podem ser acessadas globalmente) e **devem ser liberadas manualmente**
- **Alocação mais lenta** que na stack
- Utiliza uma **estrutura de dados do tipo heap**. Os blocos são **alocados sob demanda na heap**
- Pode sofrer fragmentação após sucessivas alocações e liberações de memória
- **Os dados são acessados por ponteiros**
- A alocação pode falhar se muita memória for solicitada
- Adequada para se utilizar quando **não se sabe exatamente o quanto de espaço será alocado** antes de compilar, **ou esse espaço a alocar for muito grande**
- Responsável por vazamentos de memória :(

Pilha (Stack) versus Heap

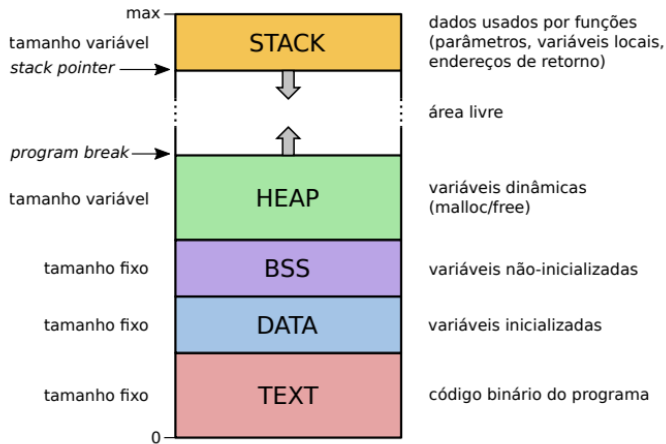
Comparativo entre alocação estática e dinâmica de memória:

Alocação estática (stack)	Alocação dinâmica (heap)
Armazenado na memória RAM	Armazenado na memória RAM
Variáveis são liberadas automaticamente no final do escopo	Variáveis não dependem do escopo (podem ser acessadas globalmente) e devem ser liberadas manualmente
Alocação mais rápida que na heap	Alocação mais lenta que na stack
Implementado usando uma estrutura de dados do tipo pilha	Blocos de dados são alocados sob demanda
Armazena dados locais e endereços de retorno utilizados na passagem de parâmetros	Pode sofrer fragmentação após sucessivas alocações e liberações de memória
Os dados podem ser usados sem ponteiros	Os dados são acessados por ponteiros
Pode ocorrer estouro de pilha quando a pilha é muito usada	A alocação pode falhar se muita memória é solicitada
É usada quando se sabe exatamente o quanto de espaço será alocado antes do tempo de compilação e esse espaço não é muito grande	É usada quando não se sabe exatamente o quanto de espaço será alocado antes do tempo de compilação ou esse espaço é muito grande
Geralmente possui um tamanho máximo predeterminado quando o programa inicia	Responsável por vazamentos de memória

Organização da Memória de um Processo- Simplificado



Organização da Memória de um Processo



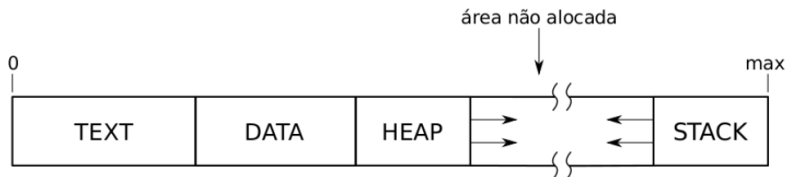
Espaços de Endereços de um Processo

Os segmentos lógicos mais importantes do espaço de endereços de um processo são (*imagem no slide anterior*):

- **Text:** código do programa e constantes. Alocado na criação do processo e inalterado até seu término.
- **Data:** Segmento de tamanho fixo, onde são alocadas as variáveis globais e estáticas.
- **Stack:** Segmento que contém a pilha de execução (stack), onde são armazenadas os parâmetros, endereços de retorno e variáveis locais de funções (nos stack frames). Varia de tamanho durante a execução.
- **Heap:** memória alocada dinamicamente, sob demanda. Varia de tamanho durante a execução do processo.

Espaços de Endereços de um Processo

Os segmentos lógicos mais importantes do espaço de endereços de um processo são:



Os próximos dois slides são de autoria do Prof. Alexandre Xavier Falcão, *Full Professor of Computer Science*, que gentilmente me autorizou a utilizá-los.

Ponteiros e Alocação Dinâmica: malloc e calloc

Na biblioteca **stdlib.h** existem duas funções para fazer alocação de memória.

- **calloc** : Nesta função são passados como parâmetro o número de blocos de memória a serem alocados e o tamanho em bytes de cada bloco.
 - ▶ Exemplo: alocar 100 inteiros:

```
int *p;  
p = calloc(100, sizeof(int));
```
- **malloc** : Nesta função é passado um único argumento, o número de bytes a serem alocados.
 - ▶ Exemplo: alocar 100 inteiros:

```
int *p;  
p = malloc(100*sizeof(int));
```
- **Diferenças**: O `calloc` *zera* todos os bits da memória alocada enquanto que o `malloc` não. Logo se não for necessário uma inicialização (com zero) da memória alocada, o `malloc` é preferível por ser um pouco mais rápido.

Ponteiros e Alocação Dinâmica: free

Juntamente com estas funções, está definida a função **free** na biblioteca **stdlib.h**.

- **free** : Esta função recebe como parâmetro um ponteiro, e libera a memória previamente alocada e apontada pelo ponteiro.

▶ Exemplo:

```
int *p;  
p = calloc(100, sizeof(int));  
....  
free(p);
```

- **Regra para uso correto de alocação dinâmica:** Toda memória alocada durante a execução de um programa deve ser desalocada (com o **free**) quando não for mais utilizada!

Referência Bibliográfica

BACKES, André. **Estrutura de dados descomplicada em linguagem C**. São Paulo: Addison Wesley, 2016.

Castro, Edson da Silva. **Uma intervenção baseada numa abordagem significativa do aprendizado de Algoritmos 1 no curso técnico de nível médio integrado em informática**. Especialização em Docência para a Educação Profissional e Tecnológica - Instituto Federal de Educação Ciência e Tecnologia de Mato Grosso do Sul, 2020.